File Handling

Tutorial

Week 7

Concepts we will Discuss

- Text File
- Concepts of File handling
- Opening and closing a file
- Reading from a file
- Writing to a file
- File directory/Absolute and Relative Path
- String Operations

What are Text Files?

- **Definition:** a type of computer file that stores the plain text information.
- **Common Formats:** Examples include .txt,.csv, .xml and many more
- **Use cases:** Easily processed ,widely used for storing configuration settings, source code and data exchange between programs
- **Advantages:** Lightweight, human-readable, and widely supported across different platforms and applications.

What is File Handling?

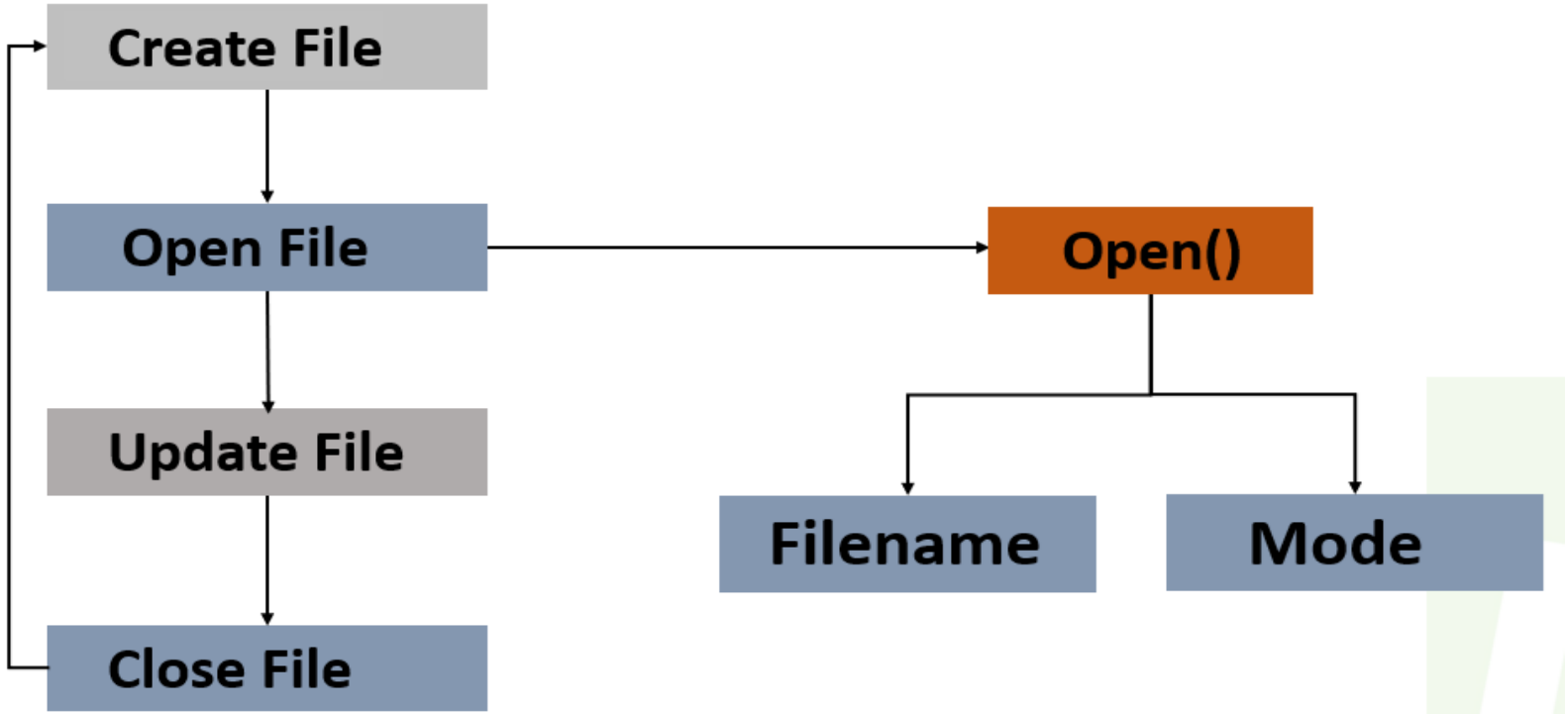
- refers to the process of working with files in a computer's file system using a programming language like Python.
- Involves operations like creating, reading, writing, updating, and deleting files.
- various function can be used **open(), close(),** with **open().**
- The mode can be 'r' for **read** mode, 'w' for **write** mode, 'a' for **append** mode, and so on.

Operations on File Handling

There are various operations that we can perform:

- **Opening a file**
- **Reading from a file**
- **Writing to a file**
- **Closing a file**
- **Appending to a file (for certain types of files)**

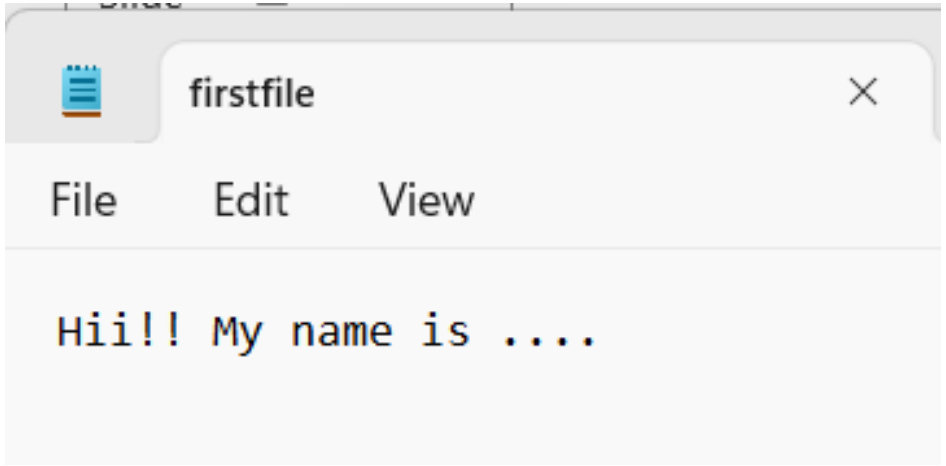
The first thing working with files is open() function



Example open()

using the open() function to open the file **'firstfile.txt'** which by default opens in a read mode.

```
f = open('firstfile.txt' , 'r')
```



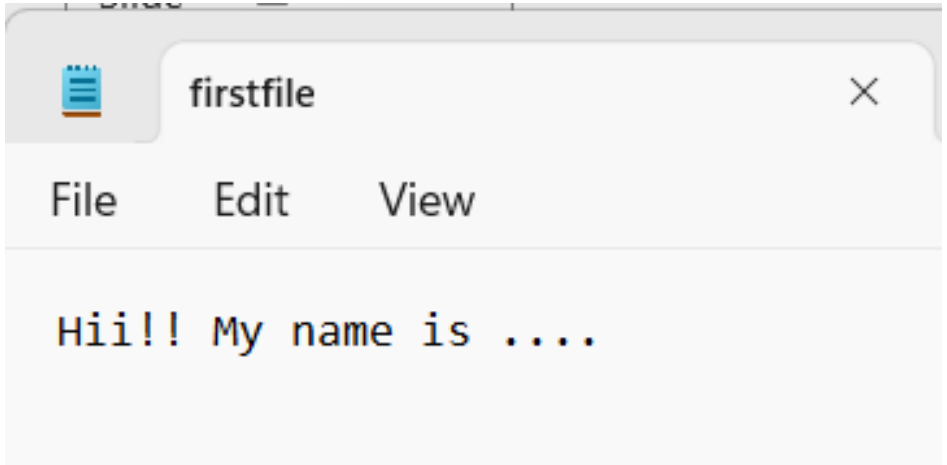
Output:

```
>>>
= RESTART: C:/Users/sandesh/De
sktop/Herald college materials
/for file handling/hello.py
>>>
```

Note: We have to use .read() to read the contents in a file

Example read()

```
f = open('firstfile.txt', 'r')
file_contents = f.read()
print(file_contents)
f.close()
```



Output:

```
f = open('firstfile.txt','r')
print(f.read())
f.close()
```

```
Hii!! My name is ....
```

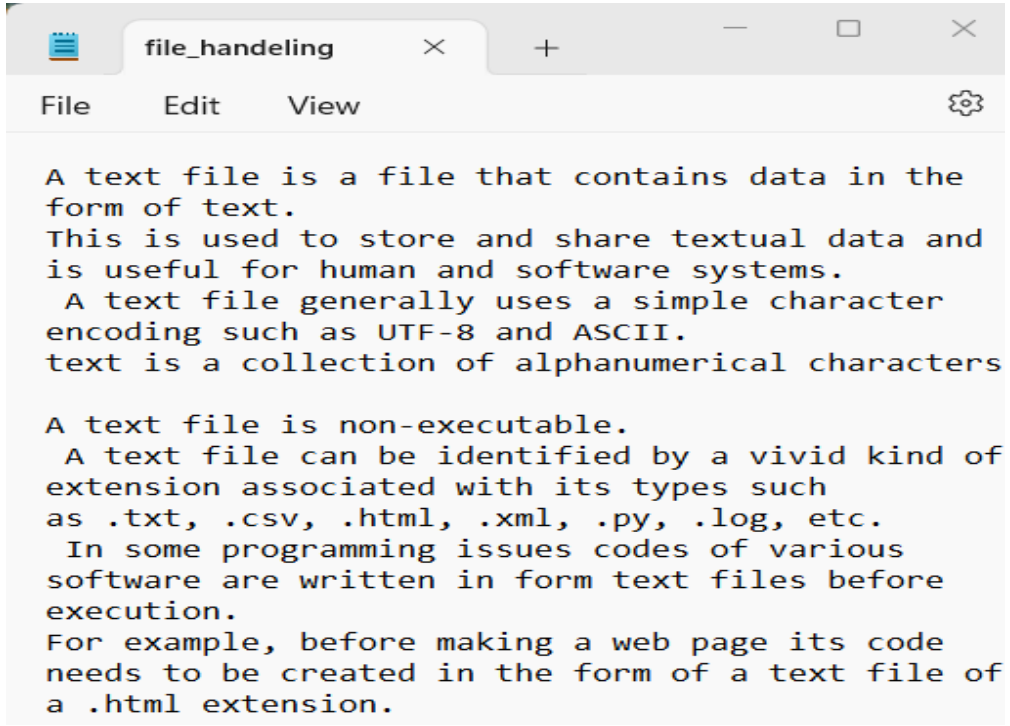
Note: Files must be closed after opening to avoid leakages, get better system performances and so on

Example readlines()

```
# Opening a file
file=open(r'file_handeling.txt', 'r')

# Iterate over each line in the file
for line in file:

# Process the data in each line
    print(line)
```

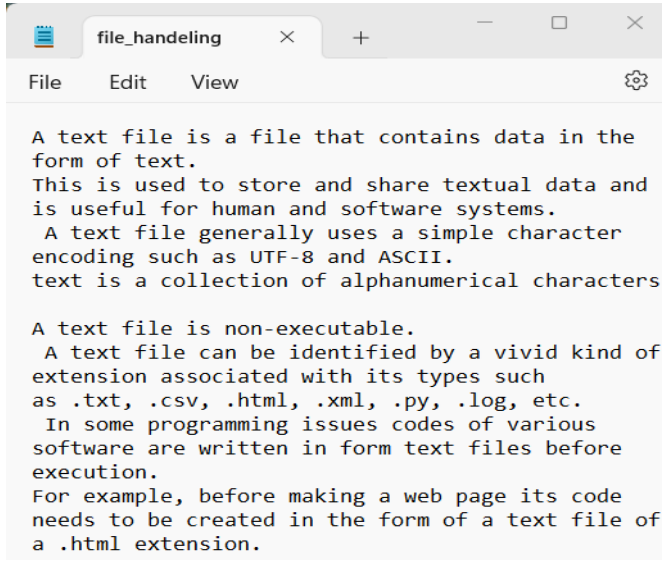


Example readlines() with Open

```
# Open the file in read mode
with open(r'file_handeling.txt', 'r') as file:

 # Iterate over each line in the file
    for line in file:

# Process the data in each line
        print(line)
```



Output: Example readlines()

```python
with open(r'C:\Users\sandesh\Desktop\file_handeling.txt', 'r') as file:

 # Iterate over each line in the file
    for line in file:

     # Process the data in each line
        print(line)
```

```
A text file is a file that contains data in the form of text.

This is used to store and share textual data and is useful for human and softwa
re systems.

 A text file generally uses a simple character encoding such as UTF-8 and ASCI
I.

text is a collection of alphanumerical characters.



A text file is non-executable.

 A text file can be identified by a vivid kind of extension associated with its
types such as .txt, .csv, .html, .xml, .py, .log, etc.

 In some programming issues codes of various software are written in form text
files before execution.
```

Handling File not found error

If the file name doesn't exist, the program will terminate with a "no such file or directory" error

```
file=open(r'abcd.txt', 'r')
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
<ipython-input-61-7384070900ff> in <module>
----> 1 file=open(r'abcd.txt', 'r')

FileNotFoundError: [Errno 2] No such file or directory: 'abcd.txt'
```

Note: This error can also occur if the file name is not found in the location looked for (uppercase and lowercase letters are treated the same for file names on Windows).

Let's check knowledge

f= open(“data.txt”)

Consider the code given above and write the answers of the following:

1. Identify the name of the file
2. Identify the name of the function used above
3. what is ‘f’ in above code
4. The above statement will ……file in ……mode.

Open / with open difference

The main difference between with open() and open():

- **with open()** automatically closes the file after completing the operations.
- **open()** requires you to manually close the file.
- Using **with open()** is considered a **safer** and **more convenient** way to handle **file operations** as it ensures that the **file is properly closed** and resources **are released**.
- It also **eliminates** the need for a **separate close()** statement, resulting in **cleaner** and more **concise** code.

Exercises

1. Write a program to read a text file and print its contents on the console.
2. Write a program to read text file line by line (guess the method to be used).
3. Write a program to read whole text file (guess the method to be used)
4. Write two different programs to read a file line by line and in a whole using with keyword.

Modes

Syntax : open(filename , mode)

- **"r"** - **Read** - Default value. Opens a file for reading, error if the file does not exist.
- **"a"** - **Append** - Opens a file for appending, creates the file if it does not exist.
- **"w"** - **Write** - Opens a file for writing, creates the file if it does not exist.
- **"x"** - **Create** - Creates the specified file, returns an error if the file exists.
- **"t"** - **Text** - Default value. Text mode.
- **"b"** - **Binary** - Binary mode (e.g., images).

Writing To a File

```
# Open file in write mode
file = open("example.txt“,"w")
# Write to file
file.write("Hello, students!")
# Close file
file.close()
```

```
# Open file in append mode
file = open("example.txt“,"a")
# Write to file
file.write("\nHow are you today?")
# Close file
file.close()
```

Note: “w” will replace texts that exist in a file and “a” helps to append texts and continue where it helps to add contents in an existing content.

Exercises

1. Write a program to open a file and write some contents inside it. It must replace the existing text inside it and add new texts. It must create a file if it doesn't really exist before.
2. Write a program to open a file and write some contents inside it. It must not replace the existing text inside it and append new texts.
3. Define a Python function called write_file() that takes two arguments, filename and text, and writes the text to the file with the given filename in write mode using the write() method.

File directory & Absolute path

- A **directory i**s a container for files and other directories
- **Directories** organize files and make them easier to find and manage.
- The **current working directory** is the directory the user is working in.
- An absolute path??

  gives path from root to the directory where your file actually exists.

Absolute Path

An **absolute path** provides the complete address of the file or directory, starting from the **root directory**. It always **begins with the root directory**, which is **represented by a forward slash (/)** on **Unix-based** systems or a **backslash (\)** on **Windows** systems. Examples of absolute paths are:

Unix-based systems: /home/user/files/myfile.txt

Windows: C:\Users\user\Documents\myfile.txt

Note: you will get the path of particular file from their properties in windows.

Exercises

1. Write a program to read a text file using an absolute path.
2. Write a Python program that reads a text file named "data.txt" and counts the number of lines in the file using a loop. The program should then print the total number of lines.

String Processing

- String processing is the manipulation of strings, which are sequences of characters, in order to extract, modify, or analyze data.

Operations

- **len(string)** - returns the length of the string
- **str(object)** - returns the string representation of the object
- **chr(number)** - returns a character from an integer ASCII code
- **ord(character)** - returns the integer ASCII code of a character

String Functions Examples

```
# Converting ASCII value to character using chr()
char = chr(65)
print(char)
# Output: A
```

```
#upper() / lower()
 my_str = "Hello World"
upper_str = my_str.upper()
lower_str = my_str.lower()
print(upper_str)
print(lower_str)

 # Output: HELLO WORLD
 # hello world
```

String Functions Examples

```
#len()
str = "Hello, World!"
print(len(str)) # Output: 13

# Converting character to ASCII value using ord()
 ascii_value = ord('A’)
print(ascii_value)
 # Output: 65
```

String Processing Methods Examples

- #Concatenating strings:

- str1 = 'Hello'
- str2 = 'World'
- result = **str1 + ' ' + str2**
- print(result)

#Splitting strings:

```
text = "The quick brown fox jumps over the lazy dog“

words = text.split()
print(words)
```

String Processing Methods Examples

```
#Reversing strings:

str = 'Hello, World!’
reversed_str = str[::-1]
print(reversed_str)
```

Note: variable name [::-1] is a slicing notation in Python [start:stop:step] :: part means to slice the entire string, from start to end, while the -1 is step.

String Processing Methods

- **string.upper()** - returns the string in all uppercase
- **string.lower()** - returns the string in all lowercase
- **string.capitalize()** - returns the string with the first character capitalized
- **string.title()** - returns the string with the first character of each word capitalized
- **string.strip()** - returns the string with leading and trailing whitespace removed
- **string.split()** - returns a list of substrings separated by a delimiter
- **string.replace()** - returns a string with all occurrences of a substring replaced by another substring
- **string.find()** - returns the index of the first occurrence of a substring in the string
- **string.startswith()** - returns True if the string starts with the specified prefix, otherwise False
- **string.endswith()** - returns True if the string ends with the specified suffix, otherwise False

String searching

- In Python, string searching refers to the process of finding the occurrence or position of a particular substring (a sequence of characters) within a given text or string.
- Python provides various methods and functions to perform string searching efficiently.

- In common, we use **find()** function to locate the first occurrence of substring within the string.
- Can be done by importing **re** module i.e.

result=re.search(word_to_search,word)

Examples

```python
text = "This is an example"
pattern = "example"
result = text.find(pattern)
print(f"Pattern found at index: {result}")
```

```
Pattern found at index: 11
```

```python
import re

text = "Python is powerful"
pattern = "power"
result = re.search(pattern, text)
print(f"Pattern found at index: {result.start()}")
```

```
Pattern found at index: 10
```

String manipulation

- String manipulation in Python involves performing various operations on strings to modify, extract, or manipulate their content.

1. Concatenation

Combining two or more strings into a single string

```
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2
print(result)
```

```
Hello World
```

2. Substring Extraction

Extracting a portion of a string

```
text = "Python is powerful"
substring = text[7:10]
print(substring)
```

```
is
```

String manipulation

3. String Length

Finding the length of a string using the len() function

```
text = "Python"
length = len(text)
print(length)
```

```
6
```

4. String Formatting

Formatting strings using placeholders or f-strings

```
name = "Alice"
age = 30
formatted_string = f"My name is {name} and I'm {age} years old."
print(formatted_string)
```

```
My name is Alice and I'm 30 years old.
```

5. String Methods

Utilizing built-in string methods for various operations

```
sentence = "Python is an easy language"
# Convert to lowercase
lowercase_sentence = sentence.lower()
# Replace 'easy' with 'powerful'
modified_sentence = lowercase_sentence.replace('easy', 'powerful')
print(modified_sentence)
```

```
python is an powerful language
```

6. String splitting and joining

Splitting string into a list of substrings or joining a list of strings into a single string.

```
csv_data = "John,Doe,30"
# Split the CSV data into a list
data_list = csv_data.split(',')
print(data_list)

# Join the list into a CSV string
csv_string = ','.join(data_list)
print(csv_string)
```

```
['John', 'Doe', '30']
John,Doe,30
```

String manipulation

7.Stripping whitespace

Removing leading and trailing whitespaces from a string.

```
input_string = "   Python is fun   "
stripped_string = input_string.strip()
print(stripped_string)
```

```
Python is fun
```

Pair Exercise

Write a Python program that reads a text file named "data.txt" and finds the longest word in the file. The program should use a loop to read each line of the file, split each line into words using the split() method, and then iterate over each word to find the longest one. Once the loop has finished, the program should print the longest word found in the file